基于 FPGA 的实时多音色合成电子乐器引擎

技 术 文 档

一、作品概述

本作品设计并实现了一套完全运行于 FPGA 内部逻辑的实时多音色音频合成引擎。系统不依赖任何处理器核或软件，音高计算、波形合成、包络调制与音频串行输出全部由硬件逻辑并行完成，面向便携式民族乐器数字化与现场演奏演示场景。

引擎采用波表合成与逐样本相位累加相结合的方式，具备 8 个独立声部，每个声部拥有独立的相位累加器、波表索引与 ADSR 幅度包络，支持多声部叠加形成和弦。音频以 16 位 PCM 通过 PT8211 数模转换器输出，采样率 46875 Hz，整条信号链的音频运算延迟低于 1 毫秒。

作品在 Tang Primer 20K 开发平台上完成验证，逻辑资源占用 8%，数字部分完全可综合、可复现，源代码公开于 GitHub。

二、赛题要求与实现对应关系

赛题要求	**本作品实现方式**
逐样本、逐声部的实时音频运算	8 个声部在 27 MHz 时钟下并行运算，每 32 个位时钟产生一个采样点，等效采样率 46875 Hz
合规合成算法	采用波表采样合成算法，内置 4 张 256 点 16 位波表（正弦、弹拨类、明亮类、风琴类）
高复音并发	8 声部独立运算并行混音，可同时发声 8 个音符
多音色切换	4 张波表可由运行参数实时切换，切换不中断发声
I2S 接口输出至 DAC	自研 PT8211 串行时序发生器，输出 BCK/WS/DIN 三线制数字音频

严格控制演奏延迟	音频链路无软件介入，合成部分延迟低于 1 ms

三、系统总体架构

系统由五个功能层构成，自上而下依次为：输入层、控制层、合成层、混音层与输出层。

层次	模块	功能
输入层	按键去抖模块	对 4 路演奏按键进行 10 ms 软件去抖，输出按下/释放脉冲
控制层	音名到相位增量映射	将音名索引换算为 24 位相位累加增量
合成层	8 × voice 模块	每声部含相位累加器、波表读取、ADSR 包络与幅度乘法
混音层	混音与饱和限幅	8 路样本右移后求和，超出满量程时饱和限幅
输出层	PT8211 时序发生器	生成 BCK/WS/DIN，帧率 46875 Hz

数据流：

按键 → 去抖 → 音名 → 相位增量 → [8 × (相位累加 → 波表 ROM → ×ADSR)] → 混音 → 饱和 → PT8211 → DAC → 功放 → 耳机

四、关键模块设计

4.1 相位累加器与波表读取

每个声部维护一个 24 位相位累加器，在每个采样周期内累加一次相位增量，高 8 位作为波表索引。相位增量按下式由目标频率计算：

$$inc = f_{note} \times 2^{24} / f_s$$

其中 f_s = 46875 Hz。以中央 C（261.63 Hz）为例，inc ≈ 93639。24 位累加器配合 8 位索引，在 256 点波表下可提供优于 0.01% 的频率精度，实测 8 个标准音全部优于 0.008%。

波表 ROM 组织为 4 个 bank × 256 点 × 16 位，共 1024 个样本，由综合工具推断为块 RAM。波表内容由 Python 脚本按谐波叠加公式预先生成，峰值幅度为满量程的 1/2，为多声部混音预留裕量。

4.2 指数 ADSR 包络

为获得接近真实拨弦乐器的听感，包络采用分段指数逼近而非线性斜坡。四个阶段分别为起音、衰减、保持与释放，每阶段均为对目标值的指数逼近：

阶段	递推关系	时间常数
起音 ATTACK	env ← env + (32767 − env) / 2^9 + 1	约 11 ms
衰减 DECAY	env ← env − (env − sustain) / 2^{12} − 1	约 87 ms
保持 SUSTAIN	env 保持于 12000（约 0.37 满量程）	持续
释放 RELEASE	env ← env − env / 2^{13} − 1	约 175 ms

指数逼近仅使用移位与加减运算，无需乘法器，8 个声部共占用的逻辑资源极小。包络值与波表样本相乘后右移 15 位完成归一化。

4.3 八声部混音与饱和限幅

八路 16 位有符号样本各自右移 2 位后送入 21 位累加器求和，最高可达 8 × 4096 = 32768，刚好覆盖满量程。求和结果经饱和判断后截断为 16 位：超出上界输出 0x7FFF，低于下界输出 0x8000，避免溢出产生的刺耳削波噪声。

4.4 PT8211 串行音频接口时序

PT8211 采用 16 位最低位对齐（Japanese Format）串行协议，与标准 I2S 不同，其字选择信号与数据位的相对位置有特定要求。本设计以 27 MHz 单时钟域实现该协议，不使用门控时钟，避免了跨时钟域问题。

参数	取值
位时钟 BCK	27 MHz ÷ 18 = 1.5 MHz
帧长	32 个位时钟（左右声道各 16 位）
采样率	1.5 MHz ÷ 32 = 46875 Hz
字选择 WS 跳变位置	位计数 3 与 19
数据加载位置	位计数 3 与 19（与 WS 边沿对齐）

设计中曾出现一个典型缺陷：数据加载点早于字选择边沿 3 个位时钟，导致 DAC 锁存时丢失高 3 位并混入另一声道数据，听感表现为严重失真。通过逐位推演时序并与厂商参考驱动对齐后修正，修正后的位序与 WS 边沿严格同步。

4.5 按键输入与鲁棒性处理

演奏按键采用 10 ms 计数去抖，输出单周期按下脉冲与释放脉冲，分别触发包络的起音与释放。针对本开发平台上存在一路空闲为低电平的输入引脚，设计中加入上电自校准：配置完成后约 30 ms 对四路输入采样一次，将已为低电平的引脚标记为不可用并屏蔽，避免其被误判为长按而产生持续发声。该机制同时保证了静置时系统完全静音。

此外，功放使能信号与包络活动状态联动：任一声部包络处于活动状态时开启功放，全部声部回到空闲并延迟约 40 ms 后关闭功放。该设计显著降低了静置时的模拟底噪。

五、工程实现

文件	说明
`src/top.v`	顶层：时钟与引脚、按键去抖、模块例化
`src/synth8.v`	八声部合成器、和弦音序器、混音限幅

src/voice.v	单声部：相位累加、波表读取、包络乘法
src/adsr.v	指数 ADSR 包络发生器
src/wavetable_rom.v	4 × 256 × 16 位波表（脚本生成）
src/note_inc.v	36 个音名到相位增量的映射表（脚本生成）
src/pt8211_tx.v	PT8211 串行音频时序发生器
src/gen_tables.py	波表与音高表的生成脚本
src/model_synth.py	位级行为模型，可离线渲染 WAV 用于验证
build.tcl	命令行综合与布局布线脚本

工程同时提供了位级 Python 模型，与 RTL 采用相同的定点数运算与截断规则，可在不下载到硬件的情况下渲染音频并核对算法正确性，用于定位"代码逻辑正确但硬件无声"类问题。

六、测试与验证

6.1 资源占用与时序

资源	占用	利用率
逻辑单元 LUT	1455 / 20736	8%
触发器 FF	434 / 16173	3%
块 RAM BSRAM	8 / 46	18%

数字信号处理单元 DSP	4 / 24	17%
时序约束	27 MHz 主时钟，布局布线后无时序违例	

6.2 功能测试

测试项	方法	结果
音频链路连通性	下载比特流，耳机接入 3.5 mm 接口	正常发声
音高准确性	位级模型渲染后做频谱分析	8 个标准音误差均小于 0.008%
复音叠加	同时按下三个演奏按键	三音叠加，无相互干扰
混音溢出	模型统计削波样本数	8 声部满幅叠加时 0 个削波样本
静置底噪	功放使能与包络联动	静置时无明显底噪
掉电保持	程序写入配置 Flash	断电重启自动加载

6.3 已知问题

- 本开发平台的板载功放存在固有嘶声，在静音时通过关闭功放使能规避，演奏过程中仍可听到轻微底噪，属模拟链路限制。
- 按键去抖窗口为 10 ms，占系统端到端响应的主要部分，后续可通过自适应去抖缩短至 3 ms 以内。
- 本平台未集成拨码开关，音色切换当前以参数形式固定，后续将通过演奏按键组合或外部串口实现运行期切换。

七、创新点

1. **全硬件并行音频架构。**八声部合成、混音与包络调制全部由逻辑门完成，不使用处理器核与软件调度，音频运算延迟低于 1 毫秒且与声部数量无关。
2. **无乘法器的指数包络。**ADSR 四个阶段均采用移位加逼近实现，单声部包络逻辑开销极小，为高复音数留出资源余量。
3. **位级行为模型辅助验证。**以 Python 复现 RTL 的定点运算与截断规则，可在硬件不可用或现象难以观察时，将问题域从"算法错误"与"硬件链路错误"中快速二分。
4. **上电引脚自校准。**在引脚电平不确定的开发平台上自动识别不可用输入并屏蔽，保证系统静置静音与演奏可靠。
5. **功放使能与包络联动。**利用包络活动状态控制功放使能，在保持演奏瞬态完整的前提下消除静置底噪。

八、后续工作

- 接入自研电容触摸与压力检测两路演奏维度，实现拨弦、按弦与力度控制；
- 在现有声部框架内引入物理建模（Karplus-Strong）拨弦合成，以极低的存储开销获得更具辨识度的音色；
- 扩展音域与音阶配置，支持五声音阶与转调；
- 补充运行期音色切换与音序回放功能。

九、参考文献

1. Sipeed. Tang Primer 20K Dock 原理图与引脚定义[Z]. 2023.
2. TM Technology. PT8211 16-bit DAC Datasheet[Z].
3. Gowin Semiconductor. GowinSynthesis 用户指南[Z]. 2023.
4. Karplus K, Strong A. Digital Synthesis of Plucked-String and Drum Timbres[J]. Computer Music Journal, 1983, 7(2): 43-55.
5. Smith J O. Physical Audio Signal Processing[EB/OL]. 2010.